

MODERN CONVOLUTIONAL NEURAL NETWORK

Dr. Nguyen Thi Kim Ngan



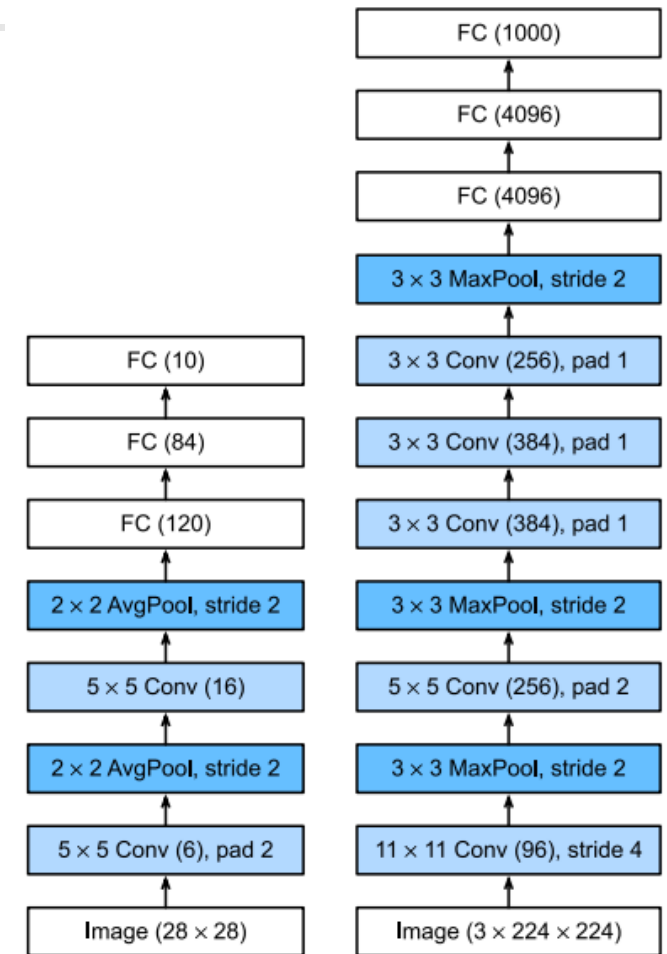
Nội dung

- Deep Convolutional Neural Networks (AlexNet)
- Networks Using Blocks (VGG)
- Network in Network (NiN)
- Multi-Branch Networks (GoogLeNet)
- Residual Networks (ResNet)
- Densely Connected Networks (DenseNet)

Deep Convolutional Neural Networks (AlexNet)

AlexNet is much deeper than LeNet-5

- AlexNet has eight layers, including:
 - 5 convolutional layers,
 - 2 fully connected hidden layers,
 - 1 fully connected output layer
- AlexNet uses the ReLU activation function instead of sigmoid
- AlexNet controls the capacity of the fully connected layers by applying dropout (whereas LeNet only uses weight decay)

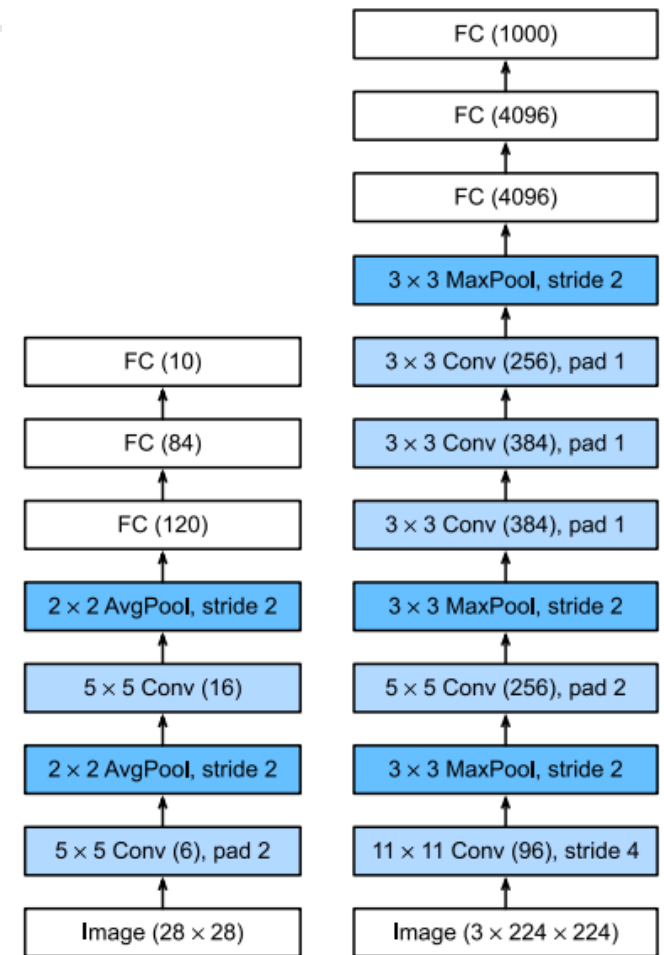


LeNet (left) and AlexNet (right)

Deep Convolutional Neural Networks (AlexNet)

```
from d2l import mxnet as d2l
from mxnet import np, npx
from mxnet.gluon import nn
npx.set_np()

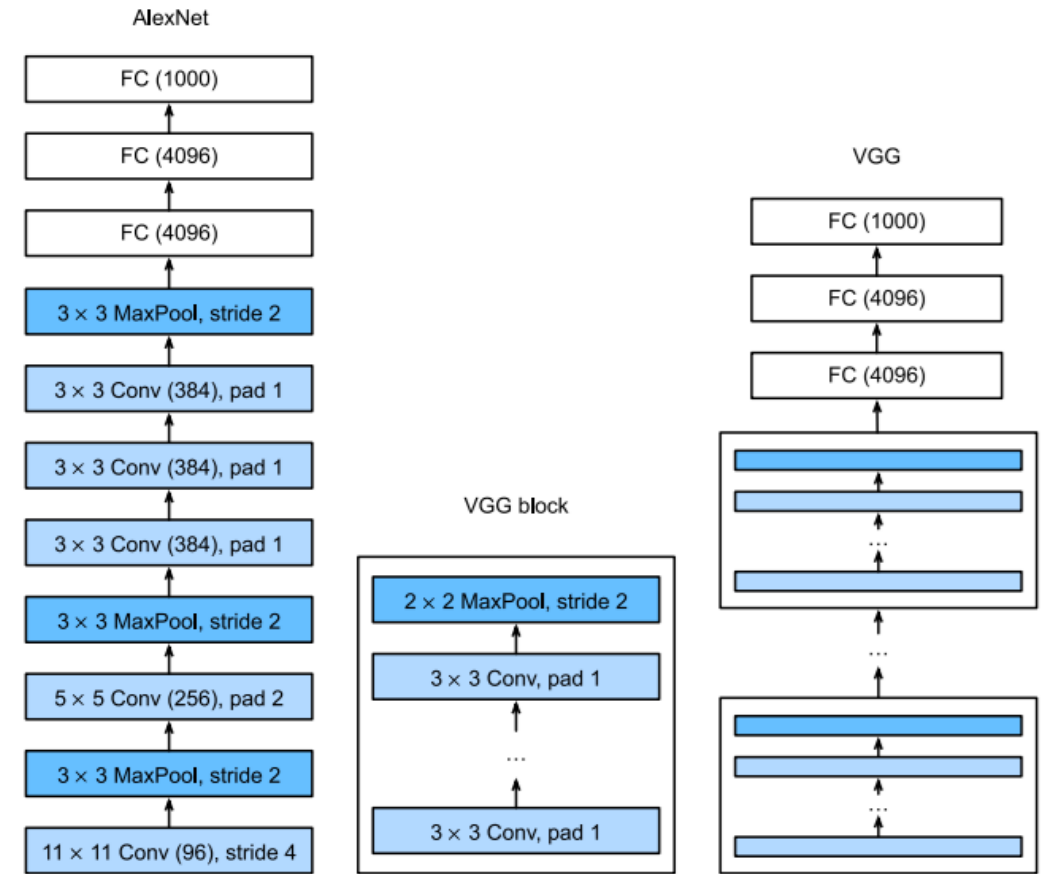
net = nn.Sequential()
# Here, we use a larger 11 x 11 window to capture objects. At the same time,
# we use a stride of 4 to greatly reduce the height and width of the output.
# Here, the number of output channels is much larger than that in LeNet
net.add(nn.Conv2D(96, kernel_size=11, strides=4, activation='relu'),
        nn.MaxPool2D(pool_size=3, strides=2),
        # Make the convolution window smaller, set padding to 2 for consistent
        # height and width across the input and output, and increase the
        # number of output channels
        nn.Conv2D(256, kernel_size=5, padding=2, activation='relu'),
        nn.MaxPool2D(pool_size=3, strides=2),
        # Use three successive convolutional layers and a smaller convolution
        # window. Except for the final convolutional layer, the number of
        # output channels is further increased. Pooling layers are not used to
        # reduce the height and width of input after the first two
        # convolutional layers
        nn.Conv2D(384, kernel_size=3, padding=1, activation='relu'),
        nn.Conv2D(384, kernel_size=3, padding=1, activation='relu'),
        nn.Conv2D(256, kernel_size=3, padding=1, activation='relu'),
        nn.MaxPool2D(pool_size=3, strides=2),
        # Here, the number of outputs of the fully connected layer is several
        # times larger than that in LeNet. Use the dropout layer to mitigate
        # overfitting
        nn.Dense(4096, activation="relu"), nn.Dropout(0.5),
        nn.Dense(4096, activation="relu"), nn.Dropout(0.5),
        # Output layer. Since we are using Fashion-MNIST, the number of
        # classes is 10, instead of 1000 as in the paper
        nn.Dense(10))
```

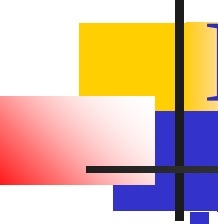


LeNet (left) and AlexNet (right)

Networks Using Blocks (VGG)

- A VGG block consists of a sequence of convolutional layers followed by a max-pooling layer to reduce the spatial dimensions
- The original VGG paper (Simonyan & Zisserman, 2014) used convolutions with 3×3 kernels and 2×2 max-pooling layers with a stride of 2 (halving the resolution after each block)





Networks Using Blocks (VGG)

- A VGG block consists of a sequence of convolutional layers followed by a max-pooling layer to reduce the spatial dimensions
- The original VGG paper (Simonyan & Zisserman, 2014) used convolutions with 3×3 kernels and 2×2 max-pooling layers with a stride of 2 (halving the resolution after each block)

```
from d2l import mxnet as d2l
from mxnet import np, npx
from mxnet.gluon import nn
npx.set_np()

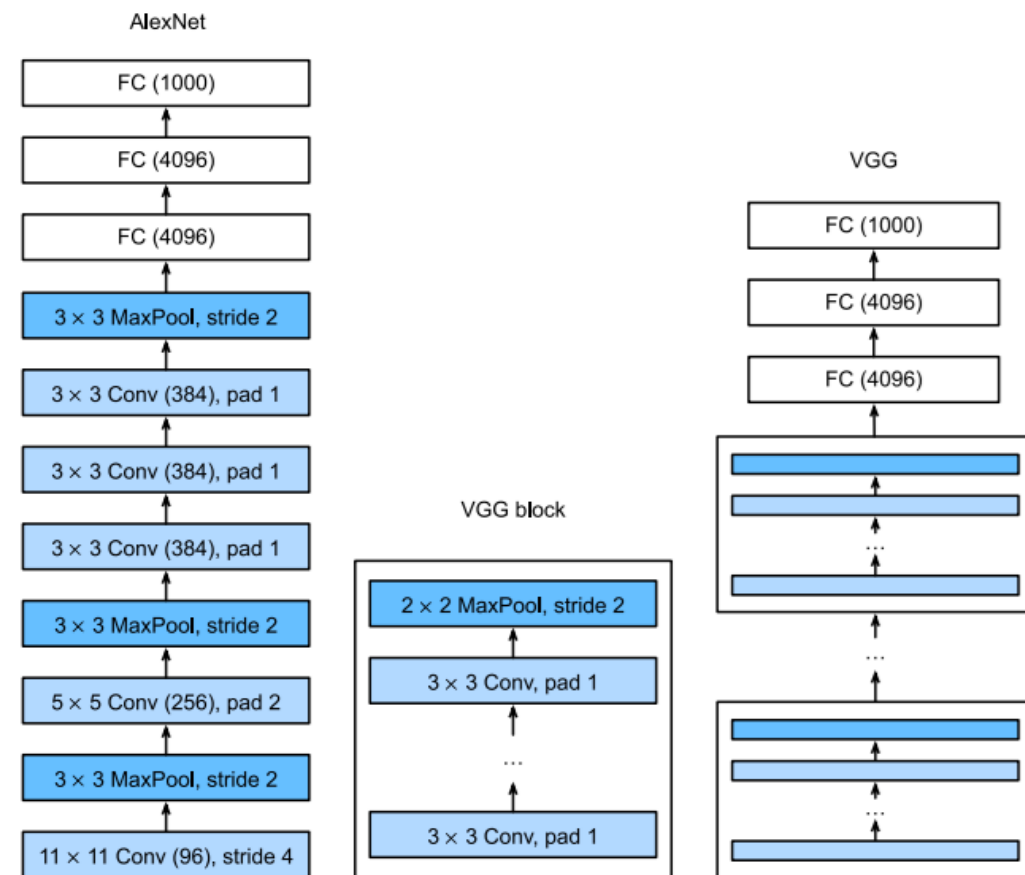
def vgg_block(num_convs, num_channels):
    blk = nn.Sequential()
    for _ in range(num_convs):
        blk.add(nn.Conv2D(num_channels, kernel_size=3,
                           padding=1, activation='relu'))
    blk.add(nn.MaxPool2D(pool_size=2, strides=2))
    return blk
```

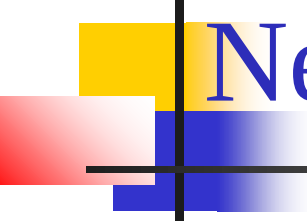
Networks Using Blocks (VGG)

The original VGG network has 5 convolutional blocks:

- The first 2 blocks each contain one convolutional layer
- The remaining 3 blocks each contain two convolutional layers
- The first block has 64 output channels, and each subsequent block doubles the number of output channels until reaching 512

The VGG network uses 8 convolutional layers and 3 fully connected layers, so it is commonly referred to as VGG-11





Networks Using Blocks (VGG)

The original VGG network has 5 convolutional blocks:

- The first 2 blocks each contain one convolutional layer
- The remaining 3 blocks each contain two convolutional layers
- The first block has 64 output channels, and each subsequent block doubles the number of output channels until reaching 512

The VGG network uses 8 convolutional layers and 3 fully connected layers, so it is commonly referred to as VGG-11

```
conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))
```

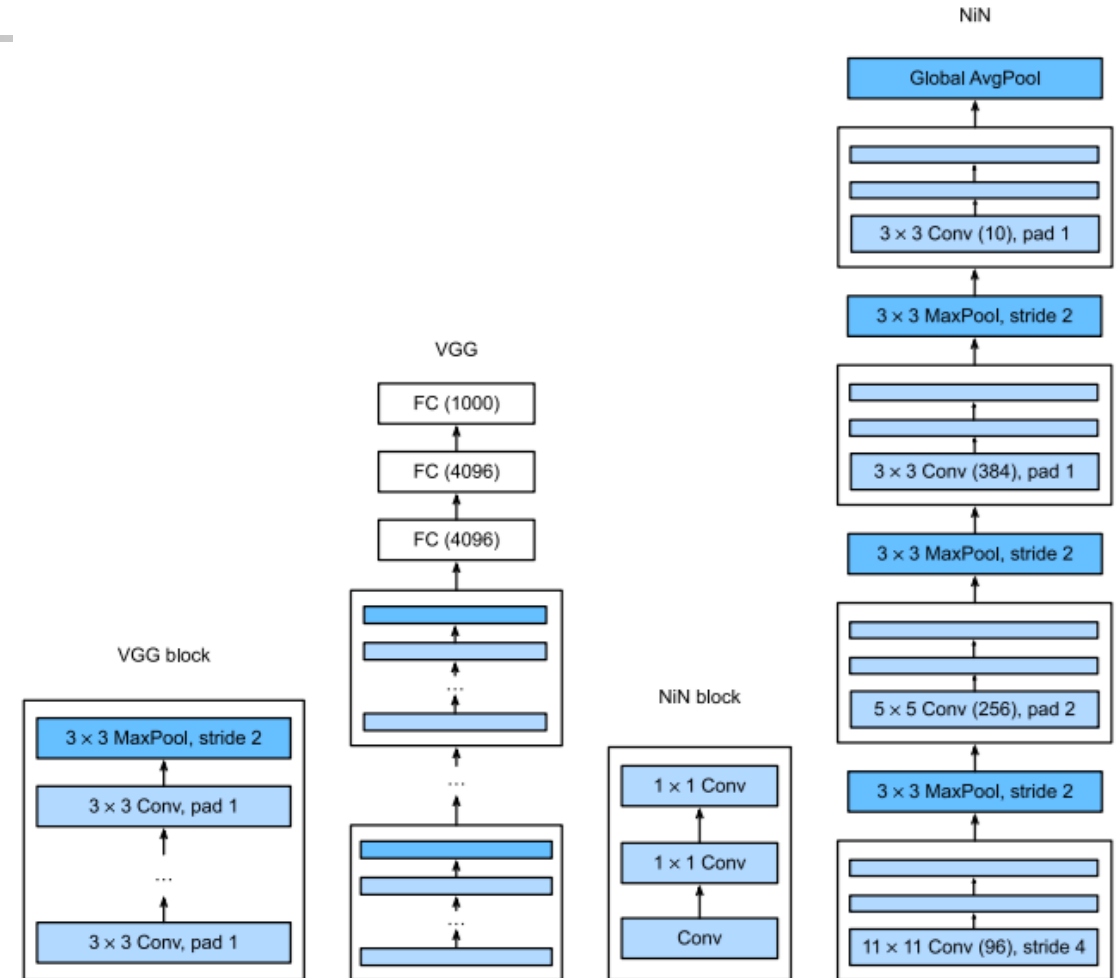
```
def vgg(conv_arch):  
    net = nn.Sequential()  
    # The convolutional layer part  
    for (num_convs, num_channels) in conv_arch:  
        net.add(vgg_block(num_convs, num_channels))  
    # The fully connected layer part  
    net.add(nn.Dense(4096, activation='relu'), nn.Dropout(0.5),  
            nn.Dense(4096, activation='relu'), nn.Dropout(0.5),  
            nn.Dense(10))  
    return net
```

Network in Network (NiN)

The network in network (NiN) blocks (Lin et al., 2013)

- (i) Use 1×1 convolutions to add local nonlinearities across the channel activations
- (ii) Use global average pooling to integrate across all locations in the last representation layer. Note that global average pooling would not be effective, were it not for the added nonlinearities

The idea behind NiN is to apply a fully connected layer at each pixel location (for each height and width). The resulting 1×1 convolution can be thought of as a fully connected layer acting independently on each pixel location

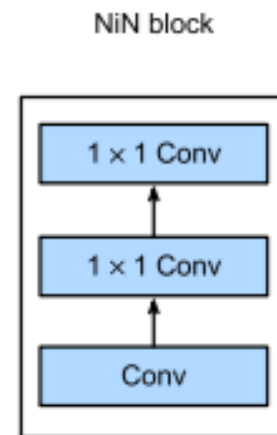


Comparing the architectures of VGG and NiN, and of their blocks.

Network in Network (NiN)

A NiN block consists of one convolutional layer followed by two $1 \times 1 \times 1$ convolutional layers, which act as fully connected layers on the pixels, and then a ReLU activation function

- The convolution window size of the first layer is usually defined by the user
- The convolution window sizes of the subsequent layers are fixed at 1×1

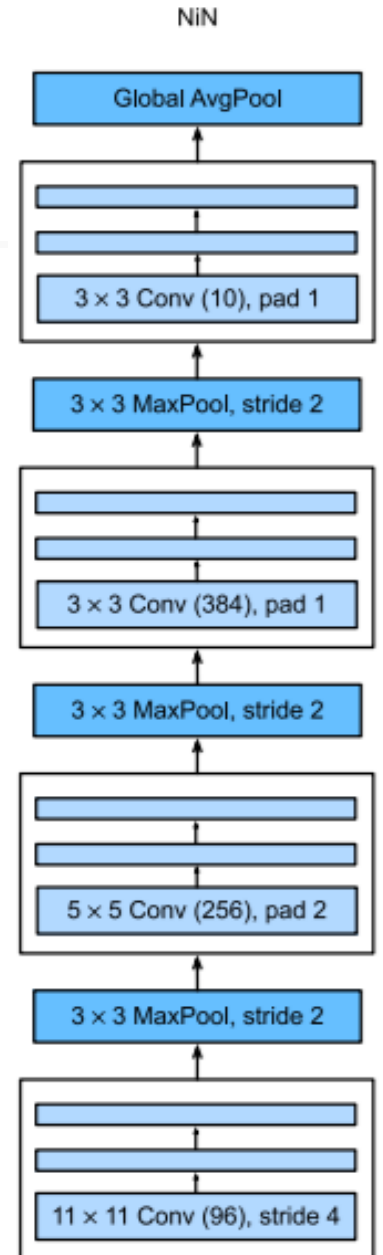


```
from d2l import mxnet as d2l
from mxnet import np, npx
from mxnet.gluon import nn
npx.set_np()

def nin_block(num_channels, kernel_size, strides, padding):
    blk = nn.Sequential()
    blk.add(nn.Conv2D(num_channels, kernel_size, strides, padding,
                      activation='relu'),
            nn.Conv2D(num_channels, kernel_size=1, activation='relu'),
            nn.Conv2D(num_channels, kernel_size=1, activation='relu'))
    return blk
```

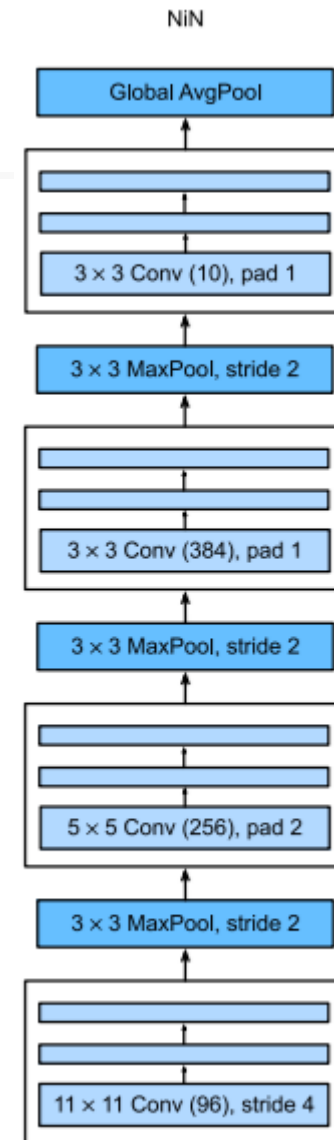
Network in Network (NiN)

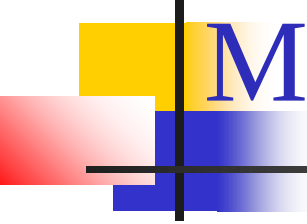
- NiN uses the same initial convolution sizes as AlexNet (it was proposed shortly thereafter). The kernel sizes are 11×11 , 5×5 , and 3×3 , respectively, and the numbers of output channels match those of AlexNet
- Each NiN block is followed by a max-pooling layer with a stride of 2 and a window shape of 3×3
- NiN block with a number of output channels equal to the number of label classes, followed by a global average pooling layer, yielding a vector of logits
- Advantages: Significantly reduces the number of parameters required by the model
- Disadvantages: Sometimes requires increased model training time



Network in Network (NiN)

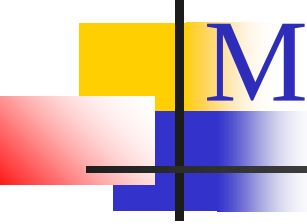
```
net = nn.Sequential()
net.add(nin_block(96, kernel_size=11, strides=4, padding=0),
        nn.MaxPool2D(pool_size=3, strides=2),
        nin_block(256, kernel_size=5, strides=1, padding=2),
        nn.MaxPool2D(pool_size=3, strides=2),
        nin_block(384, kernel_size=3, strides=1, padding=1),
        nn.MaxPool2D(pool_size=3, strides=2),
        nn.Dropout(0.5),
        # There are 10 label classes
        nin_block(10, kernel_size=3, strides=1, padding=1),
        # The global average pooling layer automatically sets the window shape
        # to the height and width of the input
        nn.GlobalAvgPool2D(),
        # Transform the four-dimensional output into two-dimensional output
        # with a shape of (batch size, 10)
        nn.Flatten())
```





Multi-Branch Networks (GoogLeNet)

- What is the optimal size of the convolutional filter?
- ⇒ In some cases, combining filters of different sizes may yield better performance



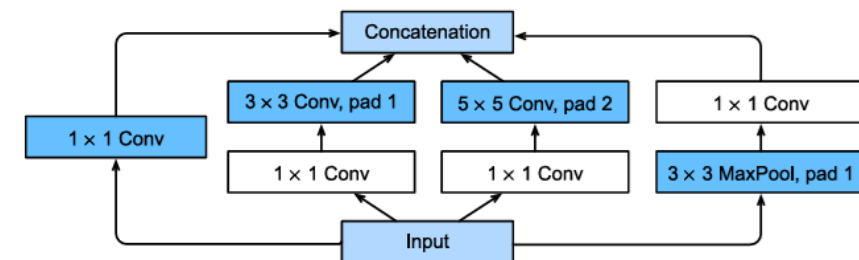
Multi-Branch Networks (GoogLeNet)

- What is the optimal size of the convolutional filter?
- ⇒ In some cases, combining filters of different sizes may yield better performance

Multi-Branch Networks (GoogLeNet)

Inception block consists of four parallel branches

- The first three branches use convolutional layers with window sizes of 1×1 , 3×3 , and 5×5 to extract information from different spatial sizes. The middle two branches also add a 1×1 convolution of the input to reduce the number of channels, reducing the model's complexity
- The fourth branch uses a 3×3 max-pooling layer, followed by a 1×1 convolutional layer to change the number of channels
- The four branches all use appropriate padding to give the input and output the same height and width
- Finally, the outputs along each branch are concatenated along the channel dimension and comprise the block's output

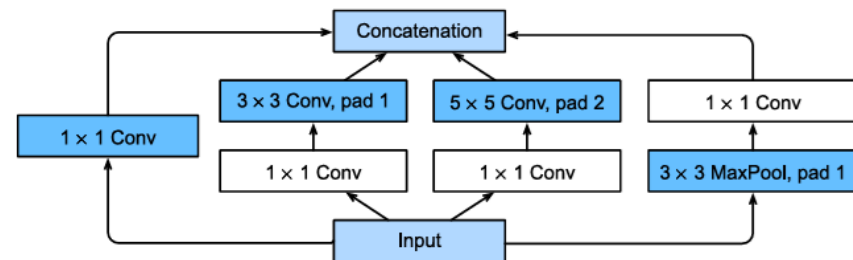


Multi-Branch Networks (GoogLeNet)

```
from d2l import mxnet as d2l
from mxnet import np, npx
from mxnet.gluon import nn
npx.set_np()

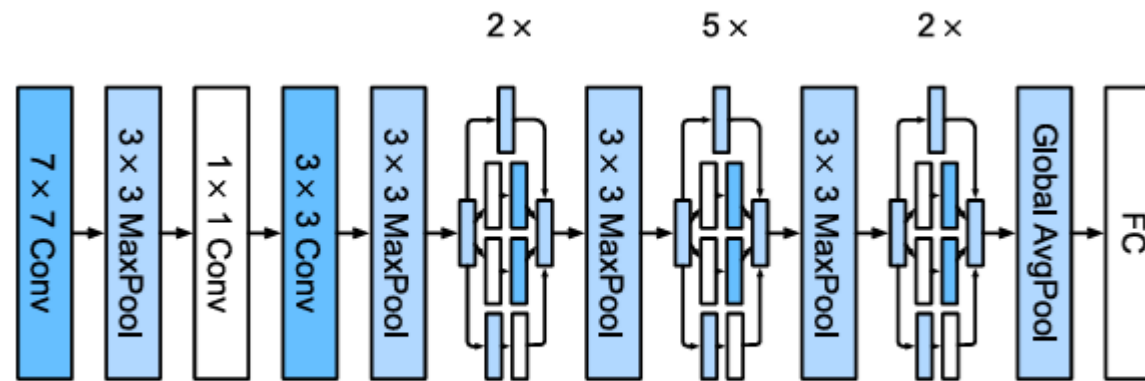
class Inception(nn.Block):
    # c1 - c4 are the number of output channels for each layer in the path
    def __init__(self, c1, c2, c3, c4, **kwargs):
        super(Inception, self).__init__(**kwargs)
        # Path 1 is a single 1 x 1 convolutional layer
        self.p1_1 = nn.Conv2D(c1, kernel_size=1, activation='relu')
        # Path 2 is a 1 x 1 convolutional layer followed by a 3 x 3
        # convolutional layer
        self.p2_1 = nn.Conv2D(c2[0], kernel_size=1, activation='relu')
        self.p2_2 = nn.Conv2D(c2[1], kernel_size=3, padding=1,
                               activation='relu')
        # Path 3 is a 1 x 1 convolutional layer followed by a 5 x 5
        # convolutional layer
        self.p3_1 = nn.Conv2D(c3[0], kernel_size=1, activation='relu')
        self.p3_2 = nn.Conv2D(c3[1], kernel_size=5, padding=2,
                               activation='relu')
        # Path 4 is a 3 x 3 maximum pooling layer followed by a 1 x 1
        # convolutional layer
        self.p4_1 = nn.MaxPool2D(pool_size=3, strides=1, padding=1)
        self.p4_2 = nn.Conv2D(c4, kernel_size=1, activation='relu')

    def forward(self, x):
        p1 = self.p1_1(x)
        p2 = self.p2_2(self.p2_1(x))
        p3 = self.p3_2(self.p3_1(x))
        p4 = self.p4_2(self.p4_1(x))
        # Concatenate the outputs on the channel dimension
        return np.concatenate((p1, p2, p3, p4), axis=1)
```

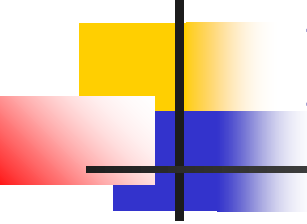


Multi-Branch Networks (GoogLeNet)

- GoogLeNet uses a stack of a total of 9 inception blocks, arranged into three groups with max-pooling in between, and global average pooling in its head to generate its estimates. Max-pooling between inception blocks reduces the dimensionality.
- At its stem, the first module is similar to AlexNet and LeNet

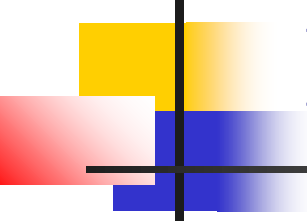


The GoogLeNet architecture.



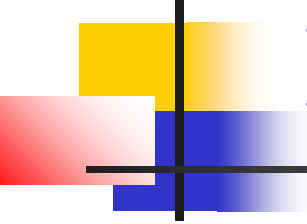
Batch Normalization

- When we train CNN, the variables in intermediate layers may take values with widely varying magnitudes: whether along the layers from input to output, across units in the same layer, and over time due to our updates to the model parameters
- We might conjecture that if one layer has variable activations that are 100 times that of another layer, this might necessitate compensatory adjustments in the learning rates



Batch Normalization

- Batch normalization is applied to individual layers, or optionally, to all of them:
In each training iteration:
 - We first normalize the inputs (of batch normalization) by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current minibatch
 - Next, we apply a scale coefficient and an offset to recover the lost degrees of freedom



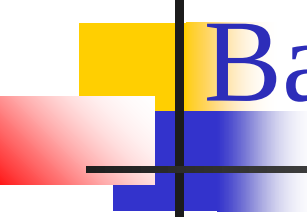
Batch Normalization

Denote by $\mathcal{B}$ a minibatch and let $\mathbf{x} \in \mathcal{B}$ be an input to batch normalization (BN). The batch normalization is defined as follows:

$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\boldsymbol{\mu}}_{\mathcal{B}}}{\hat{\boldsymbol{\sigma}}_{\mathcal{B}}} + \boldsymbol{\beta}.$$

$$\hat{\boldsymbol{\mu}}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x} \text{ and } \hat{\boldsymbol{\sigma}}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\boldsymbol{\mu}}_{\mathcal{B}})^2 + \epsilon.$$

A small constant $\epsilon > 0$ to the variance estimate to ensure that we never attempt division by zero, even in cases where the empirical variance estimate might be very small or vanish



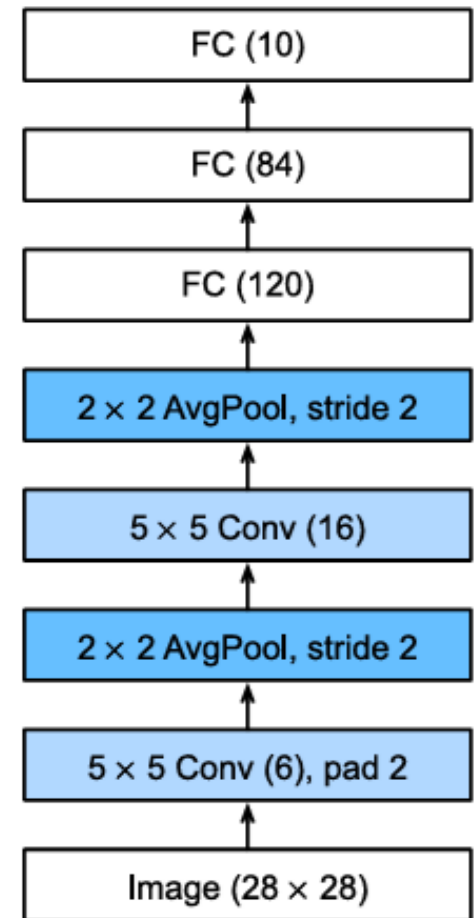
Batch Normalization

```
class BatchNorm(nn.Block):
    def __init__(self, num_features, num_dims, **kwargs):
        super(BatchNorm, self).__init__(**kwargs)
        if num_dims == 2:
            shape = (1, num_features)
        else:
            shape = (1, num_features, 1, 1)
        # The scale parameter and the shift parameter involved in gradient
        # finding and iteration are initialized to 0 and 1 respectively
        self.gamma = self.params.get('gamma', shape=shape, init=init.One())
        self.beta = self.params.get('beta', shape=shape, init=init.Zero())
        # All the variables not involved in gradient finding and iteration are
        # initialized to 0 on the CPU
        self.moving_mean = np.zeros(shape)
        self.moving_var = np.zeros(shape)

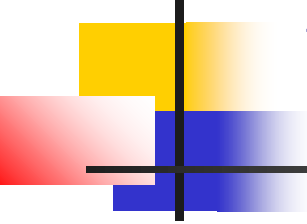
    def forward(self, X):
        # If X is not on the CPU, copy moving_mean and moving_var to the
        # device where X is located
        if self.moving_mean.ctx != X.ctx:
            self.moving_mean = self.moving_mean.copyto(X.ctx)
            self.moving_var = self.moving_var.copyto(X.ctx)
        # Save the updated moving_mean and moving_var
        Y, self.moving_mean, self.moving_var = batch_norm(
            X, self.gamma.data(), self.beta.data(), self.moving_mean,
            self.moving_var, eps=1e-12, momentum=0.9)
        return Y
```

LeNet with Batch Normalization

```
net = nn.Sequential()  
net.add(nn.Conv2D(6, kernel_size=5),  
        BatchNorm(6, num_dims=4),  
        nn.Activation('sigmoid'),  
        nn.MaxPool2D(pool_size=2, strides=2),  
        nn.Conv2D(16, kernel_size=5),  
        BatchNorm(16, num_dims=4),  
        nn.Activation('sigmoid'),  
        nn.MaxPool2D(pool_size=2, strides=2),  
        nn.Dense(120),  
        BatchNorm(120, num_dims=2),  
        nn.Activation('sigmoid'),  
        nn.Dense(84),  
        BatchNorm(84, num_dims=2),  
        nn.Activation('sigmoid'),  
        nn.Dense(10))
```



Compressed notation
for LeNet-5.



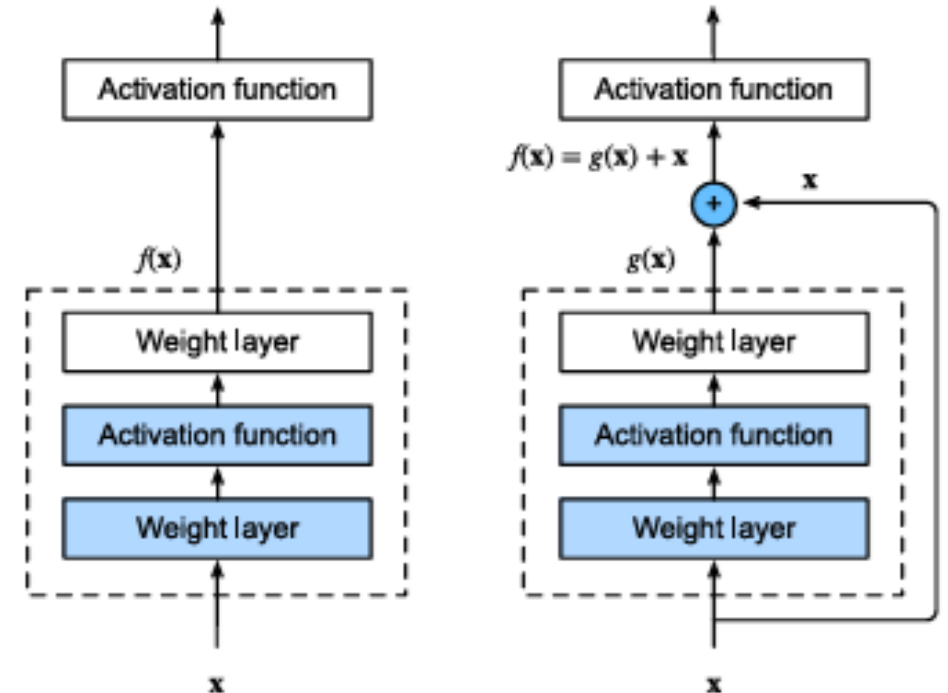
Residual Networks (ResNet)

He et al. (2016) proposed residual network (ResNet) is the idea that every additional layer should more easily contain the identity function as one of its elements

- These considerations are rather profound but they led to a surprisingly simple solution, a residual block
- ResNet won the ImageNet Large Scale Visual Recognition Challenge in 2015

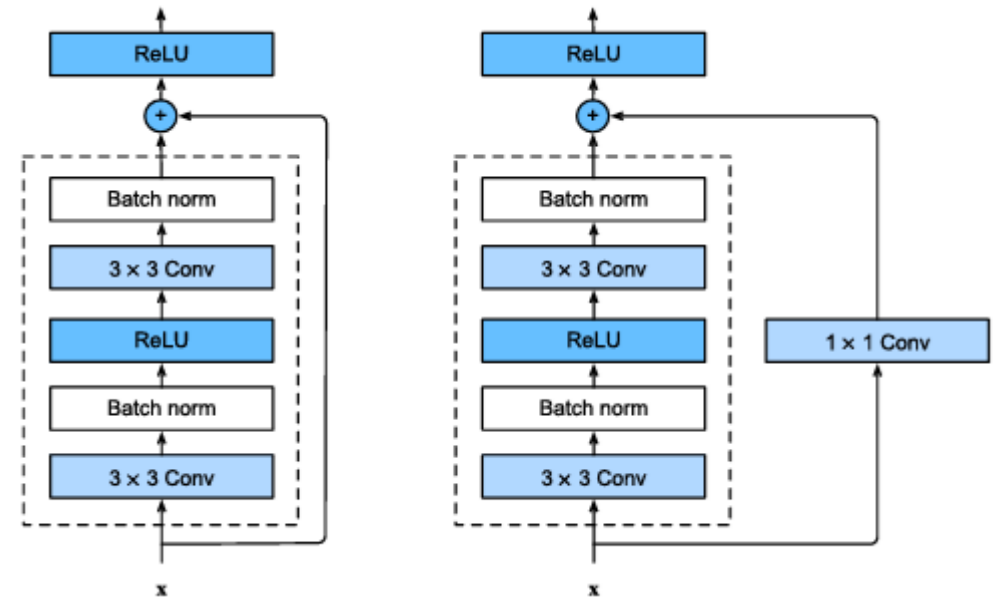
Residual Networks (ResNet)

- In a regular block (left), the portion within the dotted-line box must directly learn the mapping $f(x)$
- In a **residual block** (right), the portion within the dotted-line box needs to learn the residual mapping $g(x) = f(x) - x$, making the identity mapping $f(x) = x$ easier to learn

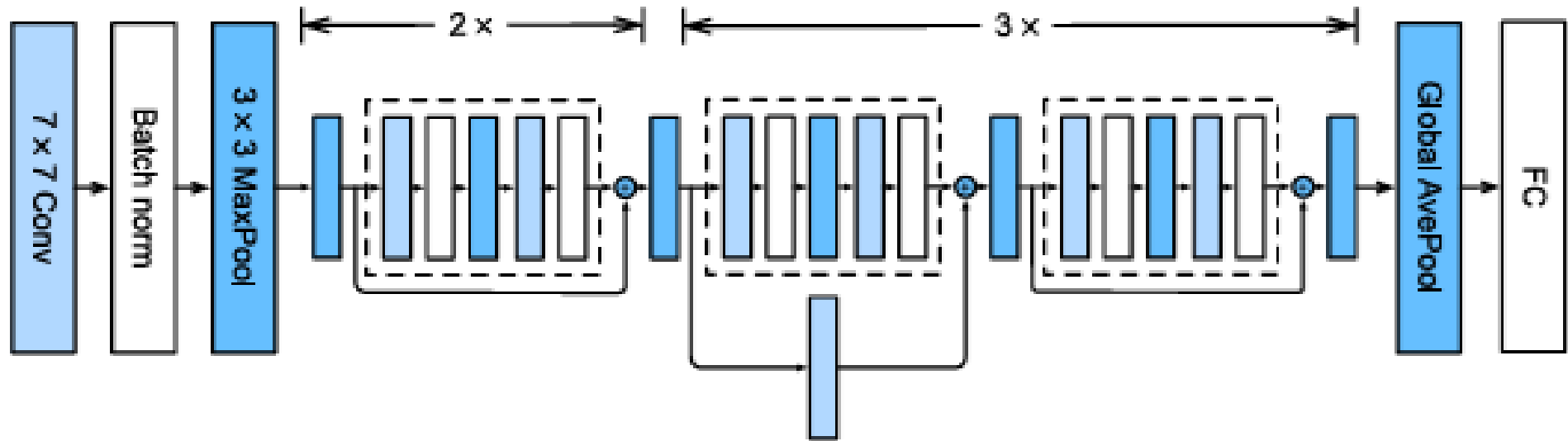


Residual Networks (ResNet)

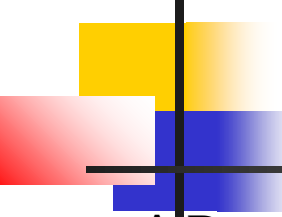
ResNet block with and without 1×1 convolution, which transforms the input into the desired shape for the addition operation



Residual Networks (ResNet)



The ResNet-18 architecture.



Densely Connected Networks (DenseNet)

A DenseNet is organized into several **Dense Blocks** connected by **Transition Layers**

1. Dense Block

- Contains multiple convolutional layers.
- Each layer receives feature maps from all previous layers.
- Encourages **feature reuse** and improves information flow.
- The number of new feature maps produced by each layer is controlled by the **growth rate** k

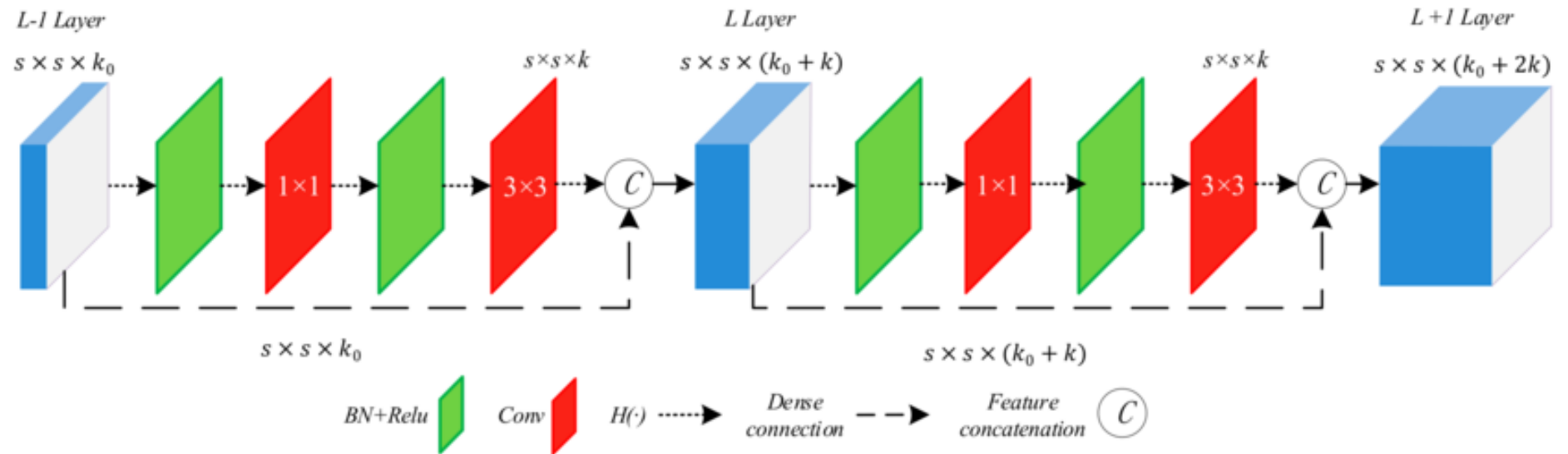
2. Transition Layer

- Connects two consecutive Dense Blocks.
- Typically consists of:
 - **Batch Normalization**
 - **1×1 Convolution**
 - **2×2 Average Pooling**
- Reduces the number of feature maps and spatial dimensions.

Overall structure:

Input → Dense Block → Transition Layer → Dense Block → Transition Layer → Dense Block → Classifier

Densely Connected Networks (DenseNet)



Densely Connected Networks (DenseNet)

